EFFTRONICS PLACEMENT PREPARATION

C Programming

Teaching handout with tested programs

Common to both tracks - CSE Software and ECE/EEE Firmware

How to use this handout

- C is a **DEEP** subject for both tracks. The written paper is pen-and-paper, and the interview questions you on what you wrote.
- For every program: read the **logic** line, write the code **on paper** without looking, dry-run it, then compare with the code and the **output** shown.
- The **interview follow-ups** under each program are what panels typically ask next. Answer them aloud.
- Every program here has been compiled and run with gcc (C11). Outputs are the real outputs; sizes in Modules 1, 8 and 10 are for a typical 64-bit compiler.

Contents

Module	Topic	Programs
1	Basics - data types, operators, casting	1.1 - 1.3
2	Control flow and patterns	2.1 - 2.3
3	Number logic without strings	3.1 - 3.9
4	Arrays	4.1 - 4.7
5	Strings (character arrays)	5.1 - 5.6
6	Matrices (2D arrays)	6.1 - 6.3
7	Functions and recursion	7.1 - 7.4
8	Pointers	8.1 - 8.4
9	Dynamic memory	9.1 - 9.2
10	Structures, unions and bit-fields	10.1 - 10.3
11	Storage classes and the preprocessor	11.1 - 11.2
12	Output prediction - common interview traps	12.1

Module 1: Basics - data types, operators, casting

- Know the size of each data type on a 64-bit system and that sizes are **compiler-dependent**.
- Integer division truncates; cast *before* dividing to get a decimal result.
- && and || **short-circuit**: the right side is skipped when the result is already known.

Program 1.1 - Size of data types

`File: 01_basics/sizeof_datatypes.c`

Logic: sizeof returns the size in bytes as size_t, printed with %zu.

```
#include <stdio.h>

int main(void) {
    printf("char    : %zu\n", sizeof(char));
    printf("int     : %zu\n", sizeof(int));
    printf("float   : %zu\n", sizeof(float));
    printf("double  : %zu\n", sizeof(double));
    printf("long    : %zu\n", sizeof(long));
    printf("pointer : %zu\n", sizeof(int *));
    return 0;
}
```

OUTPUT

```
char    : 1
int     : 4
float   : 4
double  : 8
long    : 8
pointer : 8
```

Interview follow-ups

- Why is sizeof a compile-time operator and not a function?
- What is the size of a pointer on a 32-bit system?

Program 1.2 - Integer division and type casting

File: 01_basics/integer_division_casting.c

Logic: 7 / 2 is integer division (3). Casting one operand to float first gives 3.50.

```c
#include <stdio.h>

int main(void) {
    int a = 7, b = 2;
    char c = 'A';

    printf("%d\n", a / b);              /* 3     */
    printf("%d\n", a % b);              /* 1     */
    printf("%.2f\n", (float)a / b);     /* 3.50  */
    printf("%.2f\n", (float)(a / b));   /* 3.00  */
    printf("%d %c\n", c, c + 2);        /* 65 C  */
    return 0;
}
```

OUTPUT

```
3
1
3.50
3.00
65 C
```

Interview follow-ups

- What is the difference between (float)a / b and (float)(a / b)?
- What does -7 % 2 give in C?

Program 1.3 - Precedence and short-circuit evaluation

File: 01_basics/precedence_short_circuit.c

Logic: * binds tighter than +. In x > 3 || y++, the left side is true, so y++ never runs.

```c
#include <stdio.h>

int main(void) {
    int x = 5, y = 10;

    printf("%d\n", x + y * 2);          /* 25 */
    printf("%d\n", (x + y) * 2);        /* 30 */
    printf("%d\n", x > 3 && y < 5);     /* 0  */
    printf("%d\n", x > 3 || y++);       /* 1  */
    printf("y = %d\n", y);              /* y is still 10 */
    return 0;
}
```

OUTPUT

```
25
30
0
1
y = 10
```

Interview follow-ups

- Why did y not become 11?
- What is the associativity of the assignment operator?

Module 2: Control flow and patterns

- Patterns test nested-loop control: the outer loop is the **row**, inner loops print **spaces** and **items**.
- Before coding, write the count of spaces and items for each row in a small table - then derive the formula.

Program 2.1 - Right-angled star triangle

`File: 02_patterns/star_triangle.c`

Logic: Row i prints i stars.

```
#include <stdio.h>

int main(void) {
    int n = 4;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++)
            printf("* ");
        printf("\n");
    }
    return 0;
}
```

OUTPUT

```
*
* *
* * *
* * * *
```

Interview follow-ups

- Print it upside down.
- Print it right-aligned.

Program 2.2 - Number pyramid

`File: 02_patterns/number_pyramid.c`

Logic: Row i has (n - i) spaces, then 1..i, then i-1..1.

```
#include <stdio.h>

int main(void) {
    int n = 4;
    for (int i = 1; i <= n; i++) {
        for (int s = 1; s <= n - i; s++) printf(" ");
        for (int j = 1; j <= i; j++)     printf("%d", j);
        for (int j = i - 1; j >= 1; j--) printf("%d", j);
        printf("\n");
    }
    return 0;
}
```

OUTPUT

```
   1
  121
 12321
1234321
```

Interview follow-ups

- How many characters does row i print in total?

Program 2.3 - Floyd's triangle

File: 02_patterns/floyds_triangle.c

Logic: A single counter keeps increasing across rows.

```
#include <stdio.h>

int main(void) {
    int n = 4, num = 1;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++)
            printf("%3d", num++);
        printf("\n");
    }
    return 0;
}
```

OUTPUT

```
  1
  2  3
  4  5  6
  7  8  9 10
```

Interview follow-ups

- What is the last number in row n? (n(n+1)/2)

Module 3: Number logic without strings

- The two core tools: **n % 10** gives the last digit, **n / 10** removes it.
- Efftronics has asked number problems that must be solved **without converting to a string**.
- Always handle edge cases out loud: 0, 1, negative numbers.

Program 3.1 - Reverse a number and check palindrome

File: 03_number_logic/reverse_palindrome_number.c

Logic: Build the reverse digit by digit: rev = rev * 10 + digit.

```
#include <stdio.h>

int reverse(int n) {
    int rev = 0;
    while (n > 0) {
        rev = rev * 10 + n % 10;
        n /= 10;
    }
    return rev;
}

int main(void) {
    int nums[] = {1234, 121, 1001, 10};
    for (int i = 0; i < 4; i++) {
        int n = nums[i];
        printf("%d -> reverse %d -> %s\n", n, reverse(n),
               reverse(n) == n ? "palindrome" : "not palindrome");
    }
    return 0;
}
```

OUTPUT

```
1234 -> reverse 4321 -> not palindrome
121 -> reverse 121 -> palindrome
1001 -> reverse 1001 -> palindrome
10 -> reverse 1 -> not palindrome
```

Interview follow-ups

- Why does reversing 10 give 1?
- How would you guard against overflow for large inputs?

Program 3.2 - Armstrong number

File: 03_number_logic/armstrong.c

Logic: Sum of each digit raised to the number of digits equals the number (153 = 1+125+27).

```
#include <stdio.h>

int countDigits(int n) {
    int c = 0;
    do { c++; n /= 10; } while (n > 0);
    return c;
}

int power(int base, int exp) {
    int r = 1;
    while (exp-- > 0) r *= base;
    return r;
}

int isArmstrong(int n) {
    int k = countDigits(n), sum = 0, t = n;
    while (t > 0) {
        sum += power(t % 10, k);
        t /= 10;
    }
    return sum == n;
}

int main(void) {
    printf("153  : %s\n", isArmstrong(153)  ? "Armstrong" : "No");
    printf("9474 : %s\n", isArmstrong(9474) ? "Armstrong" : "No");
    printf("123  : %s\n", isArmstrong(123)  ? "Armstrong" : "No");
    printf("Armstrong numbers 1-1000: ");
    for (int i = 1; i <= 1000; i++)
        if (isArmstrong(i)) printf("%d ", i);
    printf("\n");
    return 0;
}
```

OUTPUT

```
153  : Armstrong
9474 : Armstrong
123  : No
Armstrong numbers 1-1000: 1 2 3 4 5 6 7 8 9 153 370 371 407
```

Interview follow-ups

- Why use a do-while in countDigits?
- Why write our own power() instead of pow()?

Program 3.3 - Prime check and primes in a range

File: 03_number_logic/prime.c

Logic: Test divisors only up to the square root: if n = a * b, one of them is at most sqrt(n).

```
#include <stdio.h>

int isPrime(int n) {
    if (n < 2) return 0;
    for (int i = 2; i * i <= n; i++)
        if (n % i == 0) return 0;
    return 1;
}

int main(void) {
    printf("29 : %s\n", isPrime(29) ? "prime" : "not prime");
    printf("91 : %s\n", isPrime(91) ? "prime" : "not prime");
    printf("1  : %s\n", isPrime(1)  ? "prime" : "not prime");
    printf("Primes up to 50: ");
    for (int i = 1; i <= 50; i++)
        if (isPrime(i)) printf("%d ", i);
    printf("\n");
    return 0;
}
```

OUTPUT

```
29 : prime
91 : not prime
1  : not prime
Primes up to 50: 2 3 5 7 11 13 17 19 23 29 31 37 41 43 47
```

Interview follow-ups

- What is the time complexity of isPrime?
- Why is i * i <= n better than i <= sqrt(n)?

Program 3.4 - GCD and LCM

File: 03_number_logic/gcd_lcm.c

Logic: Euclid: gcd(a, b) = gcd(b, a % b). Then lcm = a / gcd * b (divide first to avoid overflow).

```
#include <stdio.h>

int gcd(int a, int b) {
    while (b != 0) {
        int r = a % b;
        a = b;
        b = r;
    }
    return a;
}

int main(void) {
    int a = 12, b = 18;
    int g = gcd(a, b);
    printf("GCD(%d, %d) = %d\n", a, b, g);
    printf("LCM(%d, %d) = %d\n", a, b, a / g * b);
    return 0;
}
```

OUTPUT

```
GCD(12, 18) = 6
LCM(12, 18) = 36
```

Interview follow-ups

- Write gcd recursively.
- Why compute a / g * b and not a * b / g?

Program 3.5 - Sum and count of digits

File: 03_number_logic/digit_sum_count.c

Logic: Same % 10 and / 10 loop, two accumulators.

```
#include <stdio.h>

int main(void) {
    int n = 50721, sum = 0, count = 0, t = n;
    while (t > 0) {
        sum += t % 10;
        count++;
        t /= 10;
    }
    printf("Number %d: digits = %d, sum = %d\n", n, count, sum);
    return 0;
}
```

OUTPUT

```
Number 50721: digits = 5, sum = 15
```

Interview follow-ups

- What does this print for n = 0? How do you fix it?

Program 3.6 - Replace every 9 with 81 (asked at Efftronics)

File: 03_number_logic/replace_nine_with_81.c

Logic: Read digits right to left. A 9 inserts two digits, so the place value jumps by 100 instead of 10.

```
#include <stdio.h>

long replaceNine(long n) {
    long result = 0, place = 1;
    if (n == 0) return 0;
    while (n > 0) {
        int d = n % 10;
        if (d == 9) {
            result += 81 * place;
            place  *= 100;
        } else {
            result += d * place;
            place  *= 10;
        }
        n /= 10;
    }
    return result;
}

int main(void) {
    printf("%ld\n", replaceNine(1298));
    printf("%ld\n", replaceNine(909));
    return 0;
}
```

OUTPUT

```
12818
81081
```

Interview follow-ups

- Dry-run 909 step by step.
- How would you replace each digit d with d*d in general?

Program 3.7 - Perfect number

File: 03_number_logic/perfect_number.c

Logic: Sum of proper divisors equals the number (6 = 1 + 2 + 3).

```
#include <stdio.h>

int isPerfect(int n) {
    int sum = 0;
    for (int i = 1; i <= n / 2; i++)
        if (n % i == 0) sum += i;
    return n > 1 && sum == n;
}

int main(void) {
    printf("Perfect numbers up to 1000: ");
    for (int i = 1; i <= 1000; i++)
        if (isPerfect(i)) printf("%d ", i);
    printf("\n");
    return 0;
}
```

OUTPUT

```
Perfect numbers up to 1000: 6 28 496
```

Interview follow-ups

- Why loop only up to n / 2?

Program 3.8 - Fibonacci series (iterative)

File: 03_number_logic/fibonacci_series.c

Logic: Keep two previous terms and slide them forward.

```
#include <stdio.h>

int main(void) {
    int n = 10, a = 0, b = 1;
    printf("First %d terms: ", n);
    for (int i = 0; i < n; i++) {
        printf("%d ", a);
        int next = a + b;
        a = b;
        b = next;
    }
    printf("\n");
    return 0;
}
```

OUTPUT

```
First 10 terms: 0 1 1 2 3 5 8 13 21 34
```

Interview follow-ups

- Compare this with the recursive version (7.2) - which is faster and why?

Program 3.9 - Decimal to binary and back (arithmetic only)

File: 03_number_logic/decimal_binary_conversion.c

Logic: Remainders of / 2 give bits from right to left; store them with a place value of 10.

```
#include <stdio.h>

long toBinary(int n) {
    long bin = 0, place = 1;
    while (n > 0) {
        bin += (n % 2) * place;
        place *= 10;
        n /= 2;
    }
    return bin;
}

int toDecimal(long bin) {
    int dec = 0, base = 1;
    while (bin > 0) {
        dec += (bin % 10) * base;
        base *= 2;
        bin /= 10;
    }
    return dec;
}

int main(void) {
    printf("13 -> %ld\n", toBinary(13));
    printf("1101 -> %d\n", toDecimal(1101));
    return 0;
}
```

OUTPUT

```
13 -> 1101
1101 -> 13
```

Interview follow-ups

- What is the largest number toBinary can handle before overflow?
- Do the same using bitwise operators.

Module 4: Arrays

- Arrays are passed to functions as a **pointer to the first element** - always pass the size too.
- Prefer in-place, single-pass solutions and state the complexity: O(n) time, O(1) extra space.

Program 4.1 - Reverse an array in place

File: 04_arrays/reverse_array.c

Logic: Two pointers from both ends, swap and move inward.

```
#include <stdio.h>

void reverse(int a[], int n) {
    int l = 0, r = n - 1;
    while (l < r) {
        int t = a[l]; a[l] = a[r]; a[r] = t;
        l++; r--;
    }
}

int main(void) {
    int a[] = {1, 2, 3, 4, 5};
    int n = sizeof(a) / sizeof(a[0]);
    reverse(a, n);
    for (int i = 0; i < n; i++) printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```

OUTPUT

```
5 4 3 2 1
```

Interview follow-ups

- Why can't reverse() compute n using sizeof itself?

Program 4.2 - Second largest in one pass

File: 04_arrays/second_largest.c

Logic: Track first and second; a new maximum pushes the old maximum down to second.

```
#include <stdio.h>
#include <limits.h>

int main(void) {
    int a[] = {12, 35, 1, 10, 34, 35};
    int n = sizeof(a) / sizeof(a[0]);
    int first = INT_MIN, second = INT_MIN;

    for (int i = 0; i < n; i++) {
        if (a[i] > first) {
            second = first;
            first = a[i];
        } else if (a[i] > second && a[i] != first) {
            second = a[i];
        }
    }
    printf("Largest = %d, second largest = %d\n", first, second);
    return 0;
}
```

OUTPUT

```
Largest = 35, second largest = 34
```

Interview follow-ups

- What should happen if all elements are equal?
- Why the a[i] != first check?

Program 4.3 - Rotate an array left by k

File: 04_arrays/rotate_array.c

Logic: Reversal method: reverse first k, reverse the rest, reverse the whole array. O(n), O(1) space.

```
#include <stdio.h>

void reverse(int a[], int l, int r) {
    while (l < r) {
        int t = a[l]; a[l] = a[r]; a[r] = t;
        l++; r--;
    }
}

void rotateLeft(int a[], int n, int k) {
    k %= n;
    reverse(a, 0, k - 1);
    reverse(a, k, n - 1);
    reverse(a, 0, n - 1);
}

int main(void) {
    int a[] = {1, 2, 3, 4, 5, 6, 7};
    int n = 7;
    rotateLeft(a, n, 2);
    for (int i = 0; i < n; i++) printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```

OUTPUT

```
3 4 5 6 7 1 2
```

Interview follow-ups

- How do you rotate right by k?
- Why k %= n?

Program 4.4 - Frequency of each element

File: 04_arrays/frequency_count.c

Logic: Count each value once using a visited marker.

```c
#include <stdio.h>

int main(void) {
    int a[] = {4, 2, 4, 5, 2, 4};
    int n = 6, visited[6] = {0};

    for (int i = 0; i < n; i++) {
        if (visited[i]) continue;
        int count = 1;
        for (int j = i + 1; j < n; j++) {
            if (a[j] == a[i]) {
                count++;
                visited[j] = 1;
            }
        }
        printf("%d occurs %d time(s)\n", a[i], count);
    }
    return 0;
}
```

OUTPUT

```
4 occurs 3 time(s)
2 occurs 2 time(s)
5 occurs 1 time(s)
```

Interview follow-ups

- This is O(n^2). How would a count array make it O(n) when values are small?

Program 4.5 - Remove duplicates from a sorted array

File: 04_arrays/remove_duplicates_sorted.c

Logic: Two pointers: k marks the end of the unique part.

```
#include <stdio.h>

int removeDuplicates(int a[], int n) {
    if (n == 0) return 0;
    int k = 1;
    for (int i = 1; i < n; i++)
        if (a[i] != a[k - 1])
            a[k++] = a[i];
    return k;
}

int main(void) {
    int a[] = {1, 1, 2, 3, 3, 3, 4};
    int k = removeDuplicates(a, 7);
    printf("Unique count = %d: ", k);
    for (int i = 0; i < k; i++) printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```

OUTPUT

```
Unique count = 4: 1 2 3 4
```

Interview follow-ups

- Why does this only work on a sorted array?

Program 4.6 - Merge two sorted arrays

File: 04_arrays/merge_sorted_arrays.c

Logic: Compare the fronts, copy the smaller, then copy whatever is left.

```
#include <stdio.h>

int main(void) {
    int a[] = {1, 4, 7, 9}, b[] = {2, 3, 8};
    int n = 4, m = 3, c[7];
    int i = 0, j = 0, k = 0;

    while (i < n && j < m)
        c[k++] = (a[i] <= b[j]) ? a[i++] : b[j++];
    while (i < n) c[k++] = a[i++];
    while (j < m) c[k++] = b[j++];

    for (int x = 0; x < k; x++) printf("%d ", c[x]);
    printf("\n");
    return 0;
}
```

OUTPUT

```
1 2 3 4 7 8 9
```

Interview follow-ups

- This is the merge step of which sorting algorithm?

Program 4.7 - Find the missing number from 1 to n

File: 04_arrays/missing_number.c

Logic: Expected sum n(n+1)/2 minus the actual sum gives the missing value.

```
#include <stdio.h>

int main(void) {
    int a[] = {1, 2, 4, 5, 6};          /* numbers 1..6, one missing */
    int n = 6, sum = 0;
    for (int i = 0; i < n - 1; i++) sum += a[i];
    printf("Missing number = %d\n", n * (n + 1) / 2 - sum);
    return 0;
}
```

OUTPUT

```
Missing number = 3
```

Interview follow-ups

- How would you solve it with XOR to avoid overflow?

Module 5: Strings (character arrays)

- A C string is a char array ending with **'\0'**. Every loop stops at '\0'.
- Interviewers often ask you to write strlen, strcpy, strcmp yourself - **no <string.h>**.
- ASCII facts to remember: 'A' = 65, 'a' = 97, '0' = 48, and lower - upper = 32.

Program 5.1 - Write strlen, strcpy and strcmp yourself

File: 05_strings/my_strlen_strcpy_strcmp.c

Logic: Walk the characters until '\0'. strcmp returns the difference at the first mismatch.

```
#include <stdio.h>

int myStrlen(const char *s) {
    int n = 0;
    while (s[n] != '\0') n++;
    return n;
}

void myStrcpy(char *dst, const char *src) {
    int i = 0;
    while ((dst[i] = src[i]) != '\0') i++;
}

int myStrcmp(const char *a, const char *b) {
    int i = 0;
    while (a[i] != '\0' && a[i] == b[i]) i++;
    return (unsigned char)a[i] - (unsigned char)b[i];
}

int main(void) {
    char copy[20];
    myStrcpy(copy, "Efftronics");
    printf("copy = %s, length = %d\n", copy, myStrlen(copy));
    printf("cmp(apple, apply) = %d\n", myStrcmp("apple", "apply"));
    printf("cmp(abc, abc)     = %d\n", myStrcmp("abc", "abc"));
    return 0;
}
```

OUTPUT

```
copy = Efftronics, length = 10
cmp(apple, apply) = -20
cmp(abc, abc)     = 0
```

Interview follow-ups

- Why is the source parameter declared const char *?
- What happens if the destination array is too small?

Program 5.2 - Reverse a string in place

File: 05_strings/reverse_string.c

Logic: Same two-pointer swap as arrays; find the length first.

```
#include <stdio.h>

void reverseStr(char s[]) {
    int r = 0;
    while (s[r] != '\0') r++;
    r--;
    for (int l = 0; l < r; l++, r--) {
        char t = s[l]; s[l] = s[r]; s[r] = t;
    }
}

int main(void) {
    char s[] = "embedded";
    reverseStr(s);
    printf("%s\n", s);
    return 0;
}
```

OUTPUT

```
deddebme
```

Interview follow-ups

- Why must s be declared as char s[] and not char *s = "embedded"?

Program 5.3 - Palindrome string

File: 05_strings/palindrome_string.c

Logic: Compare characters from both ends.

```
#include <stdio.h>

int isPalindrome(const char s[]) {
    int r = 0;
    while (s[r] != '\0') r++;
    for (int l = 0, k = r - 1; l < k; l++, k--)
        if (s[l] != s[k]) return 0;
    return 1;
}

int main(void) {
    printf("madam : %s\n", isPalindrome("madam") ? "yes" : "no");
    printf("level : %s\n", isPalindrome("level") ? "yes" : "no");
    printf("hello : %s\n", isPalindrome("hello") ? "yes" : "no");
    return 0;
}
```

OUTPUT

```
madam : yes
level : yes
hello : no
```

Interview follow-ups

- Make it ignore case and spaces ("Never odd or even").

Program 5.4 - Count vowels, consonants and words

File: 05_strings/vowel_consonant_word_count.c

Logic: A word starts where a non-space follows a space (or the start).

```
#include <stdio.h>

int main(void) {
    char s[] = "Efftronics builds smart IoT products";
    int vowels = 0, consonants = 0, words = 0, inWord = 0;

    for (int i = 0; s[i] != '\0'; i++) {
        char c = s[i];
        if (c >= 'A' && c <= 'Z') c += 32;          /* to lowercase */
        if (c >= 'a' && c <= 'z') {
            if (c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u')
                vowels++;
            else
                consonants++;
        }
        if (s[i] != ' ' && !inWord) { words++; inWord = 1; }
        else if (s[i] == ' ')       inWord = 0;
    }
    printf("Vowels = %d, Consonants = %d, Words = %d\n",
           vowels, consonants, words);
    return 0;
}
```

OUTPUT

```
Vowels = 10, Consonants = 22, Words = 5
```

Interview follow-ups

- Does the word count still work with two spaces between words? Why?

Program 5.5 - Check whether two strings are anagrams

File: 05_strings/anagram.c

Logic: Count letters of the first string up, the second down; all counts must end at zero.

```
#include <stdio.h>

int isAnagram(const char *a, const char *b) {
    int count[26] = {0};
    for (int i = 0; a[i]; i++) count[a[i] - 'a']++;
    for (int i = 0; b[i]; i++) count[b[i] - 'a']--;
    for (int i = 0; i < 26; i++)
        if (count[i] != 0) return 0;
    return 1;
}

int main(void) {
    printf("listen / silent : %s\n", isAnagram("listen", "silent") ? "yes" : "no");
    printf("hello  / world  : %s\n", isAnagram("hello", "world") ? "yes" : "no");
    return 0;
}
```

OUTPUT

```
listen / silent : yes
hello  / world  : no
```

Interview follow-ups

- What does a[i] - 'a' compute?
- Time complexity compared with sorting both strings?

Program 5.6 - Toggle the case of each letter

File: 05_strings/toggle_case.c

Logic: Upper and lower case differ by 32 in ASCII.

```
#include <stdio.h>

int main(void) {
    char s[] = "Hello World 2026";
    for (int i = 0; s[i] != '\0'; i++) {
        if (s[i] >= 'a' && s[i] <= 'z')      s[i] -= 32;
        else if (s[i] >= 'A' && s[i] <= 'Z') s[i] += 32;
    }
    printf("%s\n", s);
    return 0;
}
```

OUTPUT

```
hELLO wORLD 2026
```

Interview follow-ups

- How can you toggle case with a single XOR? (s[i] ^ 32)

Module 6: Matrices (2D arrays)

- a[i][j]: i is the row, j is the column. Memory is stored **row by row** (row-major).
- Matrix multiplication of (r1 x c1) and (r2 x c2) is valid only when **c1 == r2**.

Program 6.1 - Matrix multiplication

File: 06_matrices/matrix_multiplication.c

Logic: c[i][j] = sum over k of a[i][k] * b[k][j]. Three nested loops: O(n^3).

```
#include <stdio.h>

int main(void) {
    int a[2][3] = {{1, 2, 3}, {4, 5, 6}};
    int b[3][2] = {{7, 8}, {9, 10}, {11, 12}};
    int c[2][2] = {{0}};

    for (int i = 0; i < 2; i++)
        for (int j = 0; j < 2; j++)
            for (int k = 0; k < 3; k++)
                c[i][j] += a[i][k] * b[k][j];

    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) printf("%5d", c[i][j]);
        printf("\n");
    }
    return 0;
}
```

OUTPUT

```
   58   64
  139  154
```

Interview follow-ups

- What is the size of the result of (2 x 3) times (3 x 2)?
- Why must c be initialised to zero?

Program 6.2 - Transpose of a matrix

File: 06_matrices/matrix_transpose.c

Logic: Row i becomes column i: t[j][i] = a[i][j].

```
#include <stdio.h>

int main(void) {
    int a[2][3] = {{1, 2, 3}, {4, 5, 6}};
    int t[3][2];

    for (int i = 0; i < 2; i++)
        for (int j = 0; j < 3; j++)
            t[j][i] = a[i][j];

    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 2; j++) printf("%3d", t[i][j]);
        printf("\n");
    }
    return 0;
}
```

OUTPUT

```
  1  4
  2  5
  3  6
```

Interview follow-ups

- Transpose a square matrix in place without a second array.

Program 6.3 - Sum of both diagonals

File: 06_matrices/diagonal_sum.c

Logic: Main diagonal: i == j. Secondary diagonal: i + j == n - 1.

```
#include <stdio.h>

int main(void) {
    int a[3][3] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
    int n = 3, main = 0, secondary = 0;

    for (int i = 0; i < n; i++) {
        main      += a[i][i];
        secondary += a[i][n - 1 - i];
    }
    printf("Main = %d, Secondary = %d\n", main, secondary);
    return 0;
}
```

OUTPUT

```
Main = 15, Secondary = 15
```

Interview follow-ups

- Why is this O(n) and not O(n^2)?

Module 7: Functions and recursion

- C passes arguments **by value**. To change a caller's variable, pass its **address** (pointer).
- Every recursive function needs a **base case** and must move toward it - otherwise stack overflow.

Program 7.1 - Call by value vs call by reference

File: 07_functions_recursion/swap_value_vs_reference.c

Logic: swapValue changes only its own copies; swapRef changes the originals through pointers.

```
#include <stdio.h>

void swapValue(int a, int b) { int t = a; a = b; b = t; }
void swapRef(int *a, int *b) { int t = *a; *a = *b; *b = t; }

int main(void) {
    int x = 10, y = 20;
    swapValue(x, y);
    printf("After swapValue: x = %d, y = %d\n", x, y);
    swapRef(&x, &y);
    printf("After swapRef  : x = %d, y = %d\n", x, y);
    return 0;
}
```

OUTPUT

```
After swapValue: x = 10, y = 20
After swapRef  : x = 20, y = 10
```

Interview follow-ups

- Swap two numbers without a temporary variable (using + - or XOR).

Program 7.2 - Factorial and Fibonacci (recursive)

File: 07_functions_recursion/factorial_fibonacci_recursive.c

Logic: fact(n) = n * fact(n-1); fib(n) = fib(n-1) + fib(n-2).

```
#include <stdio.h>

long fact(int n) { return (n <= 1) ? 1 : n * fact(n - 1); }
int  fib(int n)  { return (n <= 1) ? n : fib(n - 1) + fib(n - 2); }

int main(void) {
    printf("5! = %ld\n", fact(5));
    printf("fib(10) = %d\n", fib(10));
    return 0;
}
```

OUTPUT

```
5! = 120
fib(10) = 55
```

Interview follow-ups

- Why is recursive fib O(2^n)?
- How many calls does fib(5) make?

Program 7.3 - Power and sum of digits (recursive)

File: 07_functions_recursion/power_digitsum_recursive.c

Logic: power(b, e) = b * power(b, e-1); digitSum(n) = n % 10 + digitSum(n / 10).

```
#include <stdio.h>

long power(int b, int e) { return (e == 0) ? 1 : b * power(b, e - 1); }
int digitSum(int n)      { return (n == 0) ? 0 : n % 10 + digitSum(n / 10); }

int main(void) {
    printf("2^10 = %ld\n", power(2, 10));
    printf("digitSum(9875) = %d\n", digitSum(9875));
    return 0;
}
```

OUTPUT

```
2^10 = 1024
digitSum(9875) = 29
```

Interview follow-ups

- Make power() O(log e) by squaring.

Program 7.4 - Tower of Hanoi

File: 07_functions_recursion/tower_of_hanoi.c

Logic: Move n-1 disks to the helper, move the largest, then move n-1 on top. Total moves = 2^n - 1.

```
#include <stdio.h>

void hanoi(int n, char from, char to, char via) {
    if (n == 0) return;
    hanoi(n - 1, from, via, to);
    printf("Move disk %d from %c to %c\n", n, from, to);
    hanoi(n - 1, via, to, from);
}

int main(void) {
    hanoi(3, 'A', 'C', 'B');
    return 0;
}
```

OUTPUT

```
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
```

Interview follow-ups

- How many moves for 4 disks?

Module 8: Pointers

- A pointer stores an **address**. &x gives the address, *p gives the value at that address.
- p + 1 moves by **sizeof(type)** bytes, not by 1 byte.
- a[i] is exactly *(a + i). An array name decays to a pointer to its first element.

Program 8.1 - Pointer basics and pointer arithmetic

File: 08_pointers/pointer_demo.c

Logic: Moving a pointer through an array.

```
#include <stdio.h>

int main(void) {
    int a[5] = {10, 20, 30, 40, 50};
    int *p = a;

    printf("%d %d %d\n", *p, *(p + 2), p[4]);
    p++;
    printf("After p++: %d\n", *p);
    printf("Elements between a[4] and a[0]: %td\n", &a[4] - &a[0]);
    return 0;
}
```

OUTPUT

```
10 30 50
After p++: 20
Elements between a[4] and a[0]: 4
```

Interview follow-ups

- If p holds address 1000 and int is 4 bytes, what is p + 3?

Program 8.2 - Pointer to pointer

File: 08_pointers/pointer_to_pointer.c

Logic: pp holds the address of p; **pp reaches the original variable.

```
#include <stdio.h>

int main(void) {
    int x = 5;
    int *p = &x;
    int **pp = &p;

    **pp = 25;
    printf("x = %d, *p = %d, **pp = %d\n", x, *p, **pp);
    return 0;
}
```

OUTPUT

```
x = 25, *p = 25, **pp = 25
```

Interview follow-ups

- Where are double pointers used? (2D dynamic arrays, changing a pointer inside a function)

Program 8.3 - Function pointers

File: 08_pointers/function_pointer.c

Logic: A pointer can hold the address of a function - used for callbacks and lookup tables.

```
#include <stdio.h>

int add(int a, int b) { return a + b; }
int sub(int a, int b) { return a - b; }
int mul(int a, int b) { return a * b; }

int main(void) {
    int (*ops[3])(int, int) = {add, sub, mul};
    const char *names[3] = {"add", "sub", "mul"};

    for (int i = 0; i < 3; i++)
        printf("%s(8, 3) = %d\n", names[i], ops[i](8, 3));
    return 0;
}
```

OUTPUT

```
add(8, 3) = 11
sub(8, 3) = 5
mul(8, 3) = 24
```

Interview follow-ups

- Where are function pointers used in embedded systems? (interrupt vector tables)

Program 8.4 - Array decays to a pointer inside a function

File: 08_pointers/array_decay_sizeof.c

Logic: Inside the function, sizeof gives the pointer size - not the array size.

```
#include <stdio.h>

void show(int *arr) {
    printf("Inside function: sizeof = %zu\n", sizeof(arr));
}

int main(void) {
    int a[10];
    printf("In main       : sizeof = %zu\n", sizeof(a));
    show(a);
    return 0;
}
```

OUTPUT

```
In main       : sizeof = 40
Inside function: sizeof = 8
```

Interview follow-ups

- This is why every array function also takes the size n.

Module 9: Dynamic memory

- malloc: allocates, contents are garbage. calloc: allocates and sets to **zero**. realloc: resizes.
- Always check for NULL, always free, and set the pointer to NULL after freeing.
- **Memory leak** = allocated memory never freed. **Dangling pointer** = pointer used after free.

Program 9.1 - Dynamic array with malloc

File: 09_dynamic_memory/malloc_dynamic_array.c

Logic: Size decided at run time.

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int n = 5, sum = 0;
    int *arr = malloc(n * sizeof(int));
    if (arr == NULL) {
        printf("Allocation failed\n");
        return 1;
    }
    for (int i = 0; i < n; i++) {
        arr[i] = (i + 1) * (i + 1);
        sum += arr[i];
    }
    printf("Squares sum = %d\n", sum);
    free(arr);
    arr = NULL;
    return 0;
}
```

OUTPUT

```
Squares sum = 55
```

Interview follow-ups

- Why multiply by sizeof(int)?
- What happens if free(arr) is missing?

Program 9.2 - calloc vs malloc, and realloc

File: 09_dynamic_memory/calloc_realloc.c

Logic: calloc zeroes the memory; realloc keeps old values and grows the block.

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int *a = calloc(3, sizeof(int));          /* all zeros */
    if (a == NULL) return 1;
    printf("After calloc : %d %d %d\n", a[0], a[1], a[2]);

    a[0] = 1; a[1] = 2; a[2] = 3;
    int *tmp = realloc(a, 5 * sizeof(int));
    if (tmp == NULL) { free(a); return 1; }
    a = tmp;
    a[3] = 4; a[4] = 5;

    printf("After realloc:");
    for (int i = 0; i < 5; i++) printf(" %d", a[i]);
    printf("\n");
    free(a);
    return 0;
}
```

OUTPUT

```
After calloc : 0 0 0
After realloc: 1 2 3 4 5
```

Interview follow-ups

- Why store realloc's result in tmp first instead of directly in a?

Module 10: Structures, unions and bit-fields

- A **structure** gives each member its own memory; a **union** shares one block (size of the largest member).
- The compiler adds **padding** for alignment, so member order changes the structure size.
- **Bit-fields** pack small flags into bits - common in embedded register definitions.

Program 10.1 - Array of structures - find the topper

File: 10_struct_union/struct_student_topper.c

Logic: Group related data and loop over it.

```
#include <stdio.h>

struct Student {
    char name[20];
    int marks;
};

int main(void) {
    struct Student s[3] = {{"Asha", 82}, {"Ravi", 91}, {"Meena", 77}};
    int top = 0;
    for (int i = 1; i < 3; i++)
        if (s[i].marks > s[top].marks) top = i;
    printf("Topper: %s with %d marks\n", s[top].name, s[top].marks);
    return 0;
}
```

OUTPUT

```
Topper: Ravi with 91 marks
```

Interview follow-ups

- How do you access a member through a pointer? (p->marks)

Program 10.2 - Structure padding and union size

File: 10_struct_union/struct_padding_union_size.c

Logic: Same members, different order, different size. The union is only as big as its largest member.

```
#include <stdio.h>

struct A { char c; int i; char d; };
struct B { int i; char c; char d; };
union  U { int i; char c; double d; };

int main(void) {
    printf("sizeof(struct A) = %zu\n", sizeof(struct A));
    printf("sizeof(struct B) = %zu\n", sizeof(struct B));
    printf("sizeof(union U)  = %zu\n", sizeof(union U));
    return 0;
}
```

OUTPUT

```
sizeof(struct A) = 12
sizeof(struct B) = 8
sizeof(union U)  = 8
```

Interview follow-ups

- Draw the memory layout of struct A and show where the padding goes.
- Values shown are for a typical 64-bit compiler.

Program 10.3 - Bit-fields

File: 10_struct_union/bit_fields.c

Logic: Three fields share one int; each holds only as many bits as declared.

```
#include <stdio.h>

struct Flags {
    unsigned int power : 1;    /* 0 or 1  */
    unsigned int mode  : 2;    /* 0 to 3  */
    unsigned int level : 4;    /* 0 to 15 */
};

int main(void) {
    struct Flags f = {1, 3, 9};
    printf("power = %u, mode = %u, level = %u\n", f.power, f.mode, f.level);
    printf("sizeof(struct Flags) = %zu\n", sizeof(struct Flags));
    return 0;
}
```

OUTPUT

```
power = 1, mode = 3, level = 9
sizeof(struct Flags) = 4
```

Interview follow-ups

- Why can't you take the address of a bit-field member?

Module 11: Storage classes and the preprocessor

• **auto**: default local. **static** local: keeps its value between calls. **extern**: declared here, defined elsewhere. **register**: a hint to keep in a CPU register.
• Macros are plain **text substitution** before compilation - always bracket parameters and the whole body.

Program 11.1 - static local variable

File: 11_storage_preprocessor/static_variable.c

Logic: count is initialised once and remembers its value across calls.

```
#include <stdio.h>

void visit(void) {
    static int count = 0;
    int normal = 0;
    count++;
    normal++;
    printf("static = %d, normal = %d\n", count, normal);
}

int main(void) {
    visit();
    visit();
    visit();
    return 0;
}
```

OUTPUT

```
static = 1, normal = 1
static = 2, normal = 1
static = 3, normal = 1
```

Interview follow-ups

- Where is a static variable stored - stack or data segment?

Program 11.2 - The macro trap

File: 11_storage_preprocessor/macro_trap.c

Logic: SQUARE(2 + 3) expands to 2 + 3 * 2 + 3 = 11. Brackets fix it.

```
#include <stdio.h>

#define SQUARE_BAD(x)  x * x
#define SQUARE(x)      ((x) * (x))

int main(void) {
    printf("SQUARE_BAD(2 + 3) = %d\n", SQUARE_BAD(2 + 3));
    printf("SQUARE(2 + 3)     = %d\n", SQUARE(2 + 3));
    return 0;
}
```

OUTPUT

```
SQUARE_BAD(2 + 3) = 11
SQUARE(2 + 3)     = 25
```

Interview follow-ups

- What goes wrong with SQUARE(i++)?
- Macro vs inline function - when would you choose each?

Module 12: Output prediction - common interview traps

• Read every line slowly and track each variable's value in a small table.
• **Undefined behaviour:** expressions like **i++ + ++i** or **printf("%d %d", i++, i)** change i twice (or read and change it) without a sequence point. The correct answer is **'undefined behaviour - the result is compiler-dependent'**, not a number.

Program 12.1 - Mixed traps

File: 12_output_prediction/output_prediction_traps.c

Logic: Predict each line before running it.

```c
#include <stdio.h>

int main(void) {
    int n = printf("Hello\n");            /* printf returns chars printed */
    printf("n = %d\n", n);

    int i = 5;
    int j = i++ + 2;                       /* uses 5, then i becomes 6 */
    int k = ++i * 2;                       /* i becomes 7, then 7 * 2  */
    printf("i = %d, j = %d, k = %d\n", i, j, k);

    unsigned char u = 255;
    u++;                                   /* wraps around */
    printf("u = %d\n", u);

    printf("sizeof('A') = %zu\n", sizeof('A'));   /* a char literal is an int in C */

    float f = 0.1f;
    printf("%s\n", f == 0.1 ? "equal" : "not equal");

    int x;
    for (x = 0; x < 3; x++);               /* note the semicolon */
    printf("x = %d\n", x);

    switch (2) {
        case 1: printf("one ");
        case 2: printf("two ");
        case 3: printf("three ");          /* no break: falls through */
    }
    printf("\n");
    return 0;
}
```

OUTPUT

```
Hello
n = 6
i = 7, j = 7, k = 14
u = 0
sizeof('A') = 4
not equal
x = 3
two three
```

Interview follow-ups

- Explain each line of output in your own words.
- Why is 0.1f not equal to 0.1?